WORDPRESS MAINTENANCE SKILL

Projektna specifikacija | Internal brief (SL) + Developer specification (EN)

Obseg: 5 tednov / približno 200 ur • Vrsta: Claude Code Skill + majhen read-only WordPress vtičnik • Različica: 1.0 • 13. oktober 2026

Namen dokumenta: uskladiti pričakovanja naročnika in praktikanta. Specifikacija opisuje MVP; vse dodatne funkcije zahtevajo ločeno odobritev.

DEL A — INTERNA SPECIFIKACIJA (SLOVENŠČINA)

1. Poslovni cilj

Vzpostaviti ponovljiv, ročno zagnan postopek za pregled vzdrževanja WordPress strani obstoječih strank. Z enim zagonom pridobimo tehnično stanje, sledimo spremembam ter izdelamo tri poročila. Podatke uporabljamo tudi za spremljanje licenc in načrtovanje dela. Skill ne izvaja posodobitev.

2. Trije obvezni izhodi

- A. Združeno interno poročilo: pregled vseh strank, zadnji uspešni zajem, število vtičnikov in posodobitev, napake, neznani statusi licenc, ter skupna tabela vseh vtičnikov po strankah (iskanje/filtriranje v CSV).
- B. Interno poročilo za posamezno stranko: WordPress jedro, VSE nameščene teme in vtičniki (tudi neaktivni in brez posodobitev), trenutne in ponujene verzije, status aktivacije, status licence, datum zajema, opažene spremembe in potrjeni posegi.
- C. Poročilo za stranko: čist PDF s povzetkom dejansko potrjenega vzdrževanja, izvedenimi posodobitvami, odprtimi težavami in priporočili. Brez internih podatkov o licencah, geslih ali drugih strankah.

3. Pravila za licence

Status licence ni univerzalno dostopen prek standardnega WordPress REST API-ja. Zato MVP uporablja ločeno lokalno evidenco: aktivna / potekla / ni potrebna / neznano / preveri; po želji datum poteka, ponudnik, lastnik licence, datum in vir potrditve. Ne ugibamo, da je licenca aktivna zgolj zato, ker vtičnik deluje ali je posodobljen. Morebitne kasnejše integracije s ponudniki licenc so izven MVP.

4. Delovni proces

1) Naročnik ročno zažene skill za eno ali vse strani. 2) Skill varno prebere podatke in shrani časovno označen posnetek. 3) Primerja posnetke, vendar razlik ne označi avtomatično kot opravljeno delo. 4) Naročnik potrdi izvedene posege in dopolni evidence licenc. 5) Skill ustvari tri poročila; naročnik pregleda in pošlje poročilo stranki.

5. Izven obsega

Varnostne kopije, samodejno posodabljanje jedra/tem/vtičnikov, pisanje v WordPress, upravljanje prijav, nadzorna spletna aplikacija, avtomatsko pošiljanje e-pošte, SEO audit, nadzor 24/7 in univerzalno avtomatsko preverjanje vseh licenc.

6. Merila za prevzem

- Deluje na najmanj 2–3 odobrenih testnih WordPress straneh, tudi kadar ena stran ni dosegljiva.
- Interno poročilo za vsako stran vsebuje vse nameščene vtičnike in teme, ne le tistih s posodobitvami.

- Združeno poročilo vsebuje pregled po straneh in izvoz celotnega inventarja v CSV.
- Poročilo za stranko vključuje samo potrjena izvedena opravila; jasno loči ugotovitve od izvedenih posegov.
- Ob napakah in manjkajočih pravicah se status prikaže kot neznan/napaka, ne kot »vse v redu«.
- Priložena so navodila za dodajanje nove strani, upravljanje poverilnic in ponovni zagon.

DEL B — DEVELOPER SPECIFICATION (ENGLISH)

1. Project objective and boundaries

Build a manually invoked Claude Code Skill named wordpress-maintenance. It collects read-only WordPress maintenance inventory from authenticated endpoints, persists timestamped snapshots, merges optional human-maintained license and work logs, and produces exactly three report types. This is a 5-week internship MVP (~200 working hours), not a SaaS product or unattended maintenance agent.

2. Architecture and data sources

- Skill: SKILL.md with explicit workflow, scripts (Python or Node.js), report templates, configuration and tests. No interactive dashboard is required.
- WordPress: use built-in authenticated REST API where supported (e.g. /wp-json/wp/v2/plugins and /wp-json/wp/v2/themes). Implement a small custom read-only REST endpoint for core version and cached available-update metadata that are not consistently available in standard endpoints.
- Authentication: HTTPS + dedicated least-privilege account with WordPress Application Password; store secrets in environment variables or protected local secret storage, never in Git, logs, reports or prompts. Verify WordPress capability checks on every custom endpoint. Standard plugin inventory endpoints may require administrator-level capabilities; document the actual required privileges.
- Load: fetch only on manual invocation, bounded concurrency (default 2 sites), timeouts and limited retries; return cached update-transient data instead of forcing remote update checks on every request. No write operations.

3. Required inventory fields

Entity	Required fields
Site	client_id, site URL, checked_at, fetch_status, error, WordPress core installed version, available core update (if known)
Plugin (ALL)	site_id, plugin identifier/file, name, installed_version, active/inactive, update_available (true/false/unknown), offered_version (nullable), source/last_check
Theme (ALL)	theme slug, name, installed_version, active/inactive, update_available, offered_version
License register	site_id, plugin identifier, license_status, expiry_date (optional), vendor, owner, verification_source, verified_at, notes
Maintenance log	site_id, date, action, component, version_before/after (optional), confirmation_by, confirmation_at, notes

Use explicit unknown values where data cannot be verified. A missing update entry must not always be interpreted as proof of being fully up to date, especially for commercial plugins with separate update channels. Preserve raw technical JSON snapshots for auditability.

4. Report outputs and formats

- R1 - Consolidated internal: PDF summary across all configured sites + CSV inventory of every plugin on every site, including sites with no updates; allow spreadsheet filtering by plugin name, site, license status and update status.
- R2 - Per-client internal: one PDF per site containing complete plugin and theme inventory, core status, license register (with unknown clearly marked), detected changes, confirmed maintenance actions and collection errors.
- R3 - Client-facing: one branded PDF per site with period, short explanation, confirmed work performed, issues and next steps. No credentials, raw internal notes, other clients or unverified completion claims.

5. Data model and workflow

Use clients.json (non-secret site metadata), licenses.csv or licenses.json (human-verified register), maintenance_log.csv (human-confirmed work), snapshots/YYYY-MM-DD/ (raw normalized JSON), and reports/YYYY-MM/ (PDF and CSV). Keep license records stable using site_id + plugin identifier, not display name alone. Generate filenames deterministically. Provide a dry-run/sample-data mode.

Workflow: load config → authenticate → collect inventory → normalize and validate → save snapshot → compare with previous snapshot → load license register and confirmed work log → generate R1/R2/R3 → summarize failures. Changes between snapshots are observations, not proof that the agency performed the update.

6. Security, privacy and operational safeguards

- Read-only by design: no REST POST/PATCH/DELETE or update commands. No credential values in Claude context or generated reports; scripts should consume secrets directly.
- Only collect from client sites explicitly authorized by the agency. Use HTTPS, strict TLS validation, safe error handling and minimal log data.
- Restrict access to internal reports and license data. Provide sample/demo data without customer-identifying information.
- Handle unreachable sites, 401/403, disabled REST API, missing permissions, partial results, rate limits and timeouts without aborting the entire batch.

7. Five-week implementation plan

Week	Expected deliverable
1	Confirm fields and sample reports; set up test WordPress instance; investigate native REST permissions and update transients.
2	Build and secure minimal WordPress status endpoint; implement authenticated collector for core, all plugins and all themes.
3	Implement normalized snapshots, site config, license register, diffing and consolidated/per-client internal reports.
4	Implement confirmed maintenance log, branded client-facing PDF, SKILL.md and command-line invocation.
5	Test against 2–3 authorized sites, negative/error cases, documentation, handover and demo.

8. Acceptance tests

- A site with 15 plugins and zero pending updates still lists all 15 plugins in its internal report and consolidated CSV.
- An inactive plugin and an inactive theme are included. Unknown license and update states remain explicitly unknown.
- If a site returns 403 or times out, it is marked failed without preventing reports for other sites.
- Snapshot differences never appear in the client-facing report as confirmed agency work unless present in the confirmed maintenance log.
- No backup, update, or other state-changing operations are executed; secrets do not appear in files committed to Git or in reports.
- Handover includes install steps, sample configuration, endpoint installation instructions, security notes, tests and a 10-minute demo.

9. Priorities and optional stretch goals

Must have: safe data collection, full plugin/theme inventory, license register, three report types, human confirmation, documentation. Optional only if time remains: sortable HTML copy of consolidated report, expiry reminders, additional commercial-plugin license integrations, richer change history. No dashboard or automated update execution.